

Assignment

Homework1

zchenlc

A ARIN5305 Homework Assignment

October 8, 2026

Task 3 Report: DeepSeek-Assisted Fault Localization and Repair

Question 1: Which steps in your fault localization and repair procedure were significantly helped by DeepSeek Chat?

- First, I used an AI tool to draft a prompt template that describes the scenario. Taking the count-covered-buildings problem as an example, the fault localization / repair instruction was as follows:

```
> How to use: copy everything below the horizontal rule into a DeepSeek web chat (Stage 2
prescribes Instant Chat; Task 3 allows reasoning to be enabled).

---

You are an experienced Java debugging assistant. Exactly one line of the Java solution
below is wrong (only one line; every other line is a deliberate implementation idea,
written differently from other solutions). Please locate it using the failing cases I
provide.

## Problem
Count Covered Buildings (LeetCode 3531)
Link: https://leetcode.com/problems/count-covered-buildings/

Given an n×n city and a set of pairwise distinct building coordinates buildings[i] = [x, y], a
building is called covered if there is at least one other building in each of the four
directions - above, below, left and right. Return the number of covered buildings.

## The faulty code (line numbers are marked; use these line numbers when you answer)
1: import java.util.Arrays;
2: import java.util.Comparator;
3:
4: class Solution {
5:
6:     private static final int VERTICAL = 0x01, HORIZONTAL = 0x03;
7:
8:     private static final class Site {
9:         final int x;
10:        final int y;
11:        int surrounded;
12:
13:        Site(int x, int y) {
14:            this.x = x;
15:            this.y = y;
16:        }
17:    }
18:
19:    public int countCoveredBuildings(int n, int[][] buildings) {
20:        Site[] sites = new Site[buildings.length];
21:
22:        for (int i = 0; i < buildings.length; i++) {
23:            sites[i] = new Site(buildings[i][0], buildings[i][1]);
24:        }
25:
26:        // In a same-x run sorted by y, every non-endpoint building
27:        // has another building both above and below it.
28:        Arrays.sort(
29:            sites,
30:            Comparator.comparingInt((Site s) -> s.x)
31:                .thenComparingInt(s -> s.y)
```

```

32:         );
33:         markInteriorOfRuns(sites, true, VERTICAL);
34:
35:         // In a same-y run sorted by x, every non-endpoint building
36:         // has another building both left and right of it.
37:         Arrays.sort(
38:             sites,
39:             Comparator.comparingInt((Site s) -> s.y)
40:                 .thenComparingInt(s -> s.x)
41:         );
42:         markInteriorOfRuns(sites, false, HORIZONTAL);
43:
44:         int covered = 0;
45:         for (Site site : sites) {
46:             if (site.surrounded == (VERTICAL | HORIZONTAL)) {
47:                 covered++;
48:             }
49:         }
50:         return covered;
51:     }
52:
53:     private void markInteriorOfRuns(
54:         Site[] sites,
55:         boolean groupByX,
56:         int directionFlag
57:     ) {
58:         int first = 0;
59:
60:         while (first < sites.length) {
61:             int key = groupByX ? sites[first].x : sites[first].y;
62:             int afterLast = first + 1;
63:
64:             while (afterLast < sites.length) {
65:                 int nextKey = groupByX
66:                     ? sites[afterLast].x
67:                     : sites[afterLast].y;
68:
69:                 if (nextKey != key) {
70:                     break;
71:                 }
72:                 afterLast++;
73:             }
74:
75:             // first and afterLast - 1 are extrema of this row/column.
76:             for (int i = first + 1; i < afterLast - 1; i++) {
77:                 sites[i].surrounded |= directionFlag;
78:             }
79:
80:             first = afterLast;
81:         }
82:     }
83: }

```

Reproduced failing cases

- Input: n = 3, buildings = [[1,1],[2,1],[3,1]]
Expected output: 0
Actual output: **1** <- wrong result
Explanation: the three buildings all lie on the same row, so the middle one has only left and right neighbours and no neighbour above or below it; it must not be counted.

- Input: n = 6, buildings = [[1,1],[2,1],[3,1],[4,1],[5,1]]
Expected output: 0
Actual output: **3** <- wrong result
Explanation: same as above - none of the buildings in that row has neighbours both above and

```
below.
Note: the three official samples in the statement (the n=3 cross, the 2x2 block and the n=5
cross) all pass, so do not judge correctness from the samples alone.

## Please answer in the following format

1. **Line number of the fault** (use the line numbers I marked):
2. **The original line** (copy it verbatim):
3. **What is wrong and what the symptom is**: explain why this line is wrong and how it makes
   the cases above produce a wrong result.
4. **Single-line repair**: only "add one line / delete one line / modify one line" is allowed.
   Give the modified line, or state explicitly which line is added or which line is deleted.
   Do not rewrite the whole program and do not fix anything else along the way.
5. **Confidence**: how sure are you? If there are other candidate lines, list them as well and
   explain why you ruled them out.

Please write out your reasoning first, then give the final conclusion.
```

- I then queried DeepSeek Chat with reasoning enabled, recorded its answer and analysed it. Again taking count-covered-buildings as the example, the answer was as follows:

```
OK, let us reason through this step by step as requested.

---

## 1. First, understanding what the code intends to do

The code models each building as a `Site` and gives every `Site` a bit flag called `surrounded`
`:

private static final int VERTICAL = 0x01, HORIZONTAL = 0x03;

This is already a little odd:

- `VERTICAL = 0x01`, binary `0001`
- `HORIZONTAL = 0x03`, binary `0011`

The condition that later decides whether a building is covered is:

if (site.surrounded == (VERTICAL | HORIZONTAL)) {

and:

VERTICAL | HORIZONTAL = 0x01 | 0x03 = 0x03

In other words, the final requirement is `site.surrounded == 3`.

But note that `HORIZONTAL` is itself `0x03`, so it already contains the `VERTICAL` bit `0x01`.

This causes one key problem: if a building is only tagged with `HORIZONTAL`, its `surrounded`
becomes `3`, and it is therefore wrongly treated as satisfying both the vertical and the
horizontal condition.

---

## 2. Looking at the symptom through the failing case

Failing case:

n = 3, buildings = [[1,1],[2,1],[3,1]]
```

```
All three buildings are on the same row `y = 1`, with x equal to 1, 2 and 3.

### Sorting by x and marking VERTICAL

First sort by `x`, then by `y`:

- x=1: (1,1)
- x=2: (2,1)
- x=3: (3,1)

Every run of equal x has length 1.

Inside `markInteriorOfRuns`:

for (int i = first + 1; i < afterLast - 1; i++) {
    sites[i].surrounded |= directionFlag;
}

When the run length is 1:

- `first` = some position
- `afterLast = first + 1`
- the loop condition `i = first + 1; i < afterLast - 1 = first` is never satisfied.

So no building is marked `VERTICAL`.

At this point every `surrounded` is 0.

---

### Sorting by y and marking HORIZONTAL

Now sort by `y`, then by `x`:

- y=1: (1,1), (2,1), (3,1)

This is a run of length 3.

`first = 0`, `afterLast = 3`.

The loop:

for (int i = first + 1; i < afterLast - 1; i++) {
    sites[i].surrounded |= directionFlag;
}

That is `i = 1; i < 2; i++`, so only the middle building `(2,1)` is processed.

Executing:

sites[1].surrounded |= HORIZONTAL;

Because `HORIZONTAL = 0x03`, the middle building's `surrounded` goes from `0` to `3`.

---

### Final count

if (site.surrounded == (VERTICAL | HORIZONTAL)) {
    covered++;
}

`VERTICAL | HORIZONTAL = 0x01 | 0x03 = 0x03 = 3`.
```

The middle building has `surrounded == 3`, so it is counted.

The program outputs `1`, but `0` is expected.

The second case `[[1,1],[2,1],[3,1],[4,1],[5,1]]` works the same way:

- grouping by x gives runs of length 1, so nobody obtains `VERTICAL`
- grouping by y gives one run of length 5, so the middle 3 buildings obtain `HORIZONTAL = 0x03`
- in the end all three of them have `surrounded` equal to `3` and are wrongly counted

So the actual output is `3` while `0` is expected.

3. Locating the line that is actually wrong

The problem is in the constant declaration:

```
private static final int VERTICAL = 0x01, HORIZONTAL = 0x03;
```

`HORIZONTAL` should be its own bit, for example `0x02`, rather than `0x03`.

If it is changed to:

```
private static final int VERTICAL = 0x01, HORIZONTAL = 0x02;
```

then:

- `VERTICAL = 0x01`
- `HORIZONTAL = 0x02`
- `VERTICAL | HORIZONTAL = 0x03`

In that case:

- a building marked only horizontally has `surrounded = 0x02`, which is not `0x03`, so it is not counted.
- only a building marked both vertically and horizontally has `surrounded = 0x03` and is counted.

That matches the problem statement exactly.

4. Checking the other candidate lines

One might suspect:

- Is the condition on line 46 wrong?
 - It reads `site.surrounded == (VERTICAL | HORIZONTAL)`; that logic is itself correct, provided `VERTICAL` and `HORIZONTAL` are non-overlapping bits.
- Is line 77, `sites[i].surrounded |= directionFlag;`, wrong?
 - That is the standard way of setting a bit flag; it is fine.
- Is the loop boundary on line 76 wrong?
 - `for (int i = first + 1; i < afterLast - 1; i++)` skips both endpoints of each run and marks only the interior elements. For runs with equal x or equal y this correctly implements "has neighbours above and below" or "has neighbours left and right".
- Are the sort comparators on lines 30, 31, 39 and 40 wrong?
 - They sort by x then y, and by y then x, respectively; that is correct.

The only thing that can make "a building with horizontal neighbours only" count as having vertical neighbours as well is therefore that `HORIZONTAL` is defined as `0x03`, which already contains the `VERTICAL` bit.

```

---

## 5. Final conclusion

1. **Line number of the fault**: line 6

2. **The original line**:
    private static final int VERTICAL = 0x01, HORIZONTAL = 0x03;

3. **What is wrong and what the symptom is**:
    `HORIZONTAL` is defined as `0x03` while `VERTICAL` is `0x01`, so `HORIZONTAL` already
    contains the `VERTICAL` bit.
    In the failing case the three buildings are on one row: grouping by x gives nobody a `
    VERTICAL` mark, but grouping by y gives the middle building `HORIZONTAL = 0x03`.
    When the final check compares `surrounded == (VERTICAL | HORIZONTAL)`, that is `surrounded
    == 0x03`, the middle building has only left and right neighbours and no neighbours above
    or below, yet it is wrongly counted because `HORIZONTAL` carries the `VERTICAL` bit with
    it.
    Hence `n=3, [[1,1],[2,1],[3,1]]` outputs 1 and `n=6, [[1,1],[2,1],[3,1],[4,1],[5,1]]`
    outputs 3; both are wrong.

4. **Single-line repair**:
    Change line 6 to:
    private static final int VERTICAL = 0x01, HORIZONTAL = 0x02;

5. **Confidence**:
    Very high, about 99%.
    The other candidates - the check on line 46, the loops on line 76, the marking on line 77,
    the sort comparators and so on - are all consistent with the intended implementation when
    read on their own; only the bit constant on line 6 lets `HORIZONTAL` swallow `VERTICAL`,
    which precisely produces the failure "a building with horizontal neighbours only is also
    counted".

```

- Afterwards I applied the repair suggested by DeepSeek to the code and verified it by running:

```

javac -d <bin> Process\Task-3\repro\{id}\Main.java Process\Task-3\repro\patched\{id}\Solution.
java
java -cp <bin> Main

```

- Comparing DeepSeek's answers for the four problems:
 - For count-covered-buildings and minimum-swaps it produced a complete manual simulation, and both the localization and the repair were correct; the symptom it described and its analysis were also essentially correct.
 - For shortest-matching-substring its technical judgement was correct (the semantics of the right boundary of the Z-box), but it gave the line number as 88 whereas the line is actually 89. The code fragment it quoted was correct, which shows that it understood the code but did not pin down the exact line. The symptom and the analysis it gave were correct.
 - For minimum-moves, however, the line number it located was correct and its explanation was self-consistent, but the repair it proposed makes the program loop forever on official sample 3; I discuss this in detail in Question 2.

Question 2: What limitations or failure cases of LLMs did you observe?

- As described above, the LLM produced a failed repair for the minimum-moves problem.
- DeepSeek said that line 101, `if (nextPower <= strongest[nextMask][nextCell])` {}, was the problem, and the line number was correct. Its line of reasoning is plausible: when `nextPower == 0` and `strongest == 0` the state is pruned, so the terminal state that "has just used up its energy and finished collecting" never enters the queue and the function returns -1. But with the change it proposed (`<=` changed to `<`), the program no longer terminates on official sample 3, `["L.S", "RXL"]`, `energy=3`.
- Why: with `<`, states with the same remaining energy are also enqueued repeatedly; and because `R` restores energy to the maximum, "walking back to `R` and out again" produces infinitely many equivalent states, so the queue never drains. It traded a missed solution for an infinite loop, and since official sample 3 happens to contain `R`, this introduces a more serious bug.
- The version finally used changes line 101 to:

```
if (nextPower <= strongest[nextMask][nextCell] && nextPower != 0) {}
```

This version passed the verification.
- I think LLMs are still very capable at fault localization and repair, but for more complex problems they need several rounds of iterative verification, and correctness has to be checked in the actual code environment.